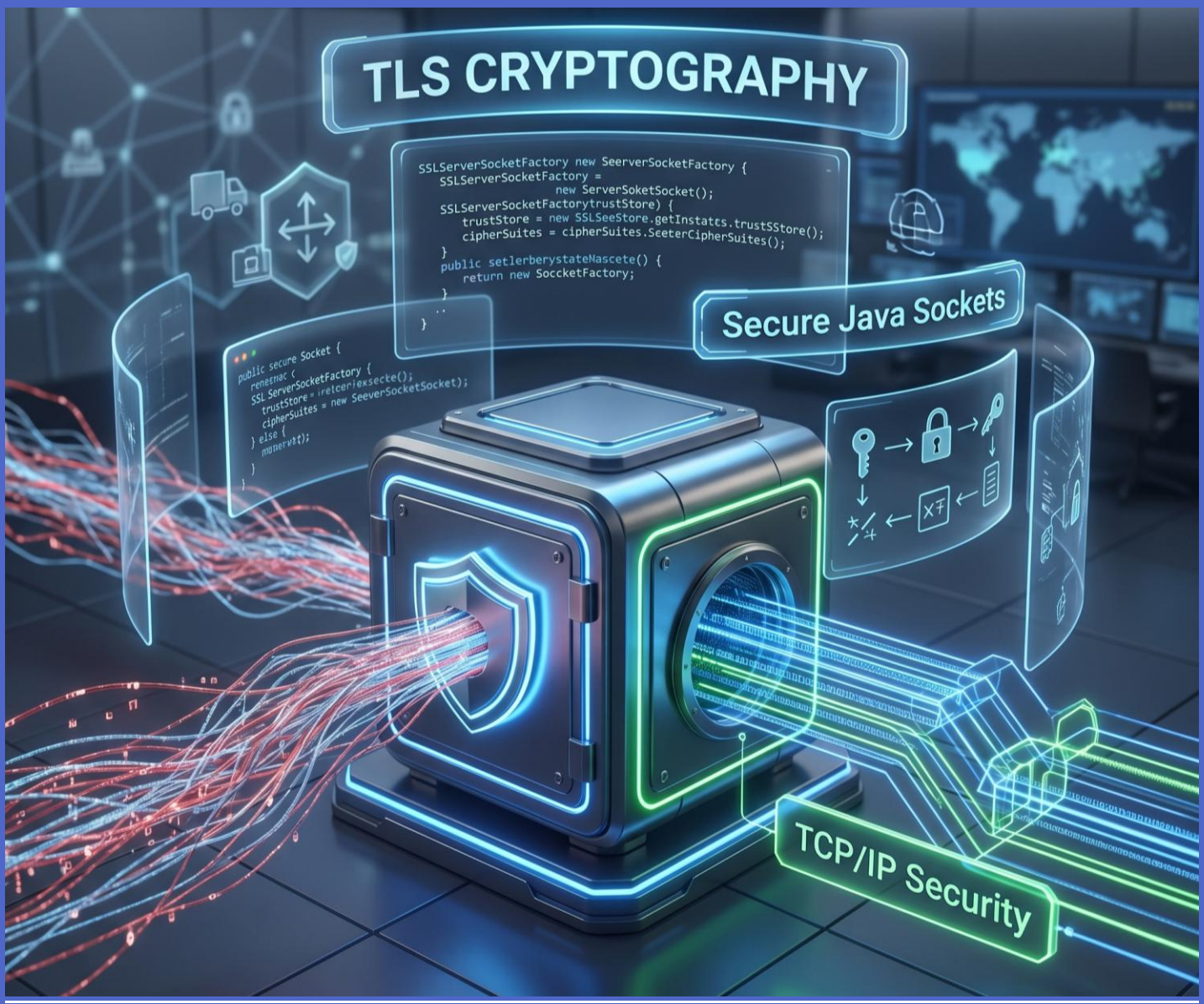
PROGRAMACION DE SERVICIOS

CASO PRACTICO I UD5



ALUMNO CESUR 25/26

Alejandro Muñoz de la Sierra

PROFESOR

Laura Bermudez Galbarriatu

I N T R O D U C C I O N

En el panorama tecnológico actual, el desarrollo de software ha dejado de ser un mero ejercicio de lógica funcional para convertirse en un desafío constante y crítico de ciberseguridad. Especialmente en entornos empresariales donde se maneja información sensible y de misión crítica —como es el caso de nuestra compañía, que transmite continuamente paquetes de datos hacia dispositivos de hardware para la gestión logística—, garantizar la confidencialidad y la integridad de las comunicaciones es un requisito arquitectónico innegociable. Bajo esta premisa, y en el marco de la Unidad Didáctica 5 sobre Técnicas de Programación Segura, nos hemos enfrentado al reto real de auditar, refactorizar y securizar un servicio de red preexistente (*legacy*).

El punto de partida de este caso práctico es un servidor de tiempo (TimeServer) fundamentado en una arquitectura cliente-servidor tradicional que opera mediante *sockets* TCP estándar. Tras realizar una auditoría de su código fuente, comprobamos que, si bien cumple perfectamente con su lógica de negocio (procesar una fecha y calcular el día de la semana correspondiente), adolece de una vulnerabilidad estructural gravísima: toda la información, tanto de subida como de bajada, viaja en texto plano. Esta carencia expone al sistema a vectores de ataque clásicos, como la interceptación de paquetes (*Packet Sniffing*) o la suplantación y alteración de datos en tránsito (*Man-in-the-Middle*).

El objetivo primordial de este proyecto ha sido mitigar esta vulnerabilidad migrando la infraestructura hacia un túnel de comunicación fuertemente cifrado utilizando el protocolo SSL/TLS. A lo largo de la presente memoria técnica, documentaremos paso a paso este proceso de ingeniería: desde el desarrollo de un cliente inseguro para comprender la topología y el flujo original de los datos, hasta la generación de una Infraestructura de Clave Pública (PKI) local mediante la herramienta keytool de Java. Todo ello con el fin último de inyectar la extensión **JSSE** (*Java Secure Sockets Extension*) y demostrar empíricamente que es posible aplicar una capa de seguridad criptográfica robusta sin necesidad de alterar ni reescribir la algoritmia central del servicio.

0 1

AUDITORÍA Y ANÁLISIS DEL SERVICIO BASE (TIMESERVER)

Para garantizar el éxito de este proyecto, la primera fase se ha centrado en realizar una auditoría técnica exhaustiva del código fuente proporcionado en la clase **TimeServer**. En ingeniería de software, comprender la algoritmia subyacente y el flujo de ejecución de un servicio heredado (***legacy***) es un paso crítico antes de proceder a desarrollar su respectivo cliente o plantear una migración de seguridad.

¿Qué hace exactamente este servicio y cómo opera? Tras desgranar el manejo de los *sockets* de red y los flujos de entrada/salida (I/O), hemos determinado que el programa actúa como un calculador remoto de fechas. Su ciclo de vida operativo se divide en las siguientes etapas:

```java
package es.cesur.dam.psp.ud5.inseguro;


// PrintStream: Lo usamos para enviar flujos de texto (la respuesta) al cliente de forma cómoda.
import java.io.PrintStream;
// ServerSocket: Es la clase fundamental para crear un servidor que escuche peticiones entrantes.
import java.net.ServerSocket;
// Socket: Representa la conexión individual o "túnel" temporal con el cliente que se acaba de conectar.
import java.net.Socket;
// Calendar y GregorianCalendar: Clases nativas de Java para manipular fechas y realizar cálculos con el calendario.
import java.util.Calendar;
import java.util.GregorianCalendar;
// Scanner: Lo utilizamos aquí para parsear (leer y dividir) los datos de entrada del cliente.
import java.util.Scanner;

```

- **Apertura y Escucha Activa:** El servidor instancia un objeto **ServerSocket** vinculado al puerto TCP 10000. Inmediatamente después, entra en un bucle infinito (while(true)), invocando el método .accept(). Esto genera un **hilo de ejecución bloqueante** (*blocking I/O*), dejando al servidor a la escucha permanente de peticiones entrantes.

```java
public class TimeServer {

    // Definimos un array constante con los nombres de los días para traducir el número que nos devuelva el calendario a texto.
    private static final String[] nombreDias = {"Domingo", "Lunes", "Martes",
                                                "Miércoles", "Jueves", "Viernes", "Sábado"};

    public static void main(String[] argv) throws Exception {
        try {
            // Instanciamos el servidor en el puerto 10000. Aquí es donde nos quedaremos "escuchando".
            ServerSocket srvSocket = new ServerSocket(10000);
            System.out.println("Servidor TimeServer (INSEGURO) a la escucha en el puerto 10000...");

```

- **Recepción y Mapeo de Datos:** Cuando un cliente establece la conexión, el servidor instancia un objeto Scanner acoplado al ***InputStream*** del socket. El protocolo de comunicación espera recibir estrictamente una secuencia de tres números enteros separados por espacios en blanco.

- **Validación de Integridad:** A través de un bucle for, el algoritmo intenta parsear el flujo de texto y almacenar los números en un *array* de tamaño 3. Si el cliente envía una cadena de caracteres alfabéticos o vulnera el formato esperado, el sistema captura la excepción, cambiando el valor de la variable booleana de control (ok) a false.

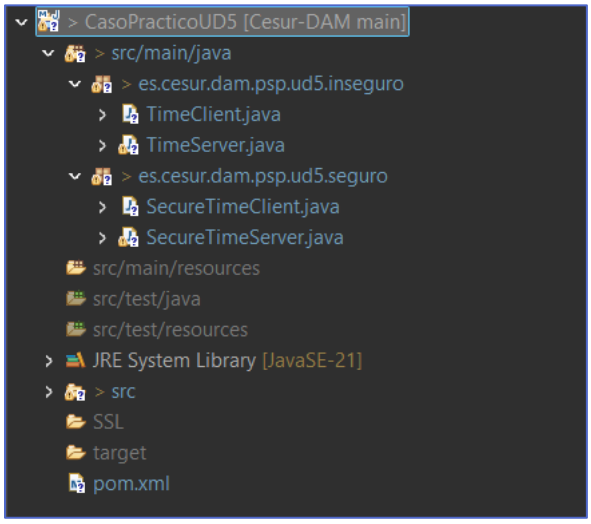
```java
            // Bucle infinito: el servidor nunca se apaga, siempre espera al siguiente cliente.
            while(true) {
                // Esta es una línea bloqueante. El hilo se detiene aquí hasta que un cliente llama a la puerta.
                Socket cliSocket = srvSocket.accept();

                // Capturamos el flujo de entrada del cliente para leer lo que nos ha enviado.
                Scanner in = new Scanner(cliSocket.getInputStream());

                // Preparamos un array de 3 posiciones para guardar: [0] Día, [1] Mes, [2] Año.
                int[] data = new int[3];
                boolean ok = true; // Bandera para controlar si el formato recibido es válido.

                // Intentamos leer exactamente 3 números enteros separados por espacios.
                for(int i = 0; i < data.length; i++) {
                    if(in.hasNextInt()) {
                        data[i]= in.nextInt(); // Guardamos el número si existe
                    } else {
                        ok = false; // Si manda una letra o faltan datos, marcamos error
                        break; // Decidimos salir del bucle para no consumir recursos innecesarios
                    }
                }

```

- **Procesamiento de la Lógica de Negocio:** Si la validación es exitosa, el servidor mapea el vector bajo el formato [Día] [Mes] [Año]. En este punto, documentamos un detalle técnico vital: el código resta una unidad al mes introducido (data[1] -= 1). Esta operación es necesaria porque la clase GregorianCalendar de Java gestiona los meses mediante un índice basado en cero (donde Enero es 0 y Diciembre es 11). Una vez ajustado, el motor calcula el día de la semana correspondiente utilizando la constante Calendar.DAY_OF_WEEK.

- **Respuesta y Cierre de Conexión:** Finalmente, el servidor busca la correspondencia en un *array* estático (nombreDias) y transmite al cliente una cadena de texto (ej. *"Este dia es Miércoles"*). En caso de fallo de validación, el servidor emite el mensaje *"Formato de datos incorrecto"*. Tras enviar la ráfaga de bytes, el servidor cierra de forma segura el socket del cliente para liberar recursos de memoria.

```java
                // Preparamos el canal de salida para contestarle al cliente.
                PrintStream out = new PrintStream(cliSocket.getOutputStream());

                if (ok) {
                    // LÍNEA CLAVE: Restamos 1 al mes. ¿Por qué? Porque la clase GregorianCalendar
                    // indexa los meses desde 0 (Enero es 0, Febrero es 1... Diciembre es 11).
                    data[1] -= 1;

                    // Creamos el objeto calendario pasándole Año, Mes (ajustado) y Día.
                    GregorianCalendar cal = new GregorianCalendar(data[2], data[1], data[0]);

                    // Obtenemos el día de la semana. Calendar.DAY_OF_WEEK devuelve 1 para Domingo, 2 para Lunes...
                    // Le restamos 1 para que coincida exactamente con los índices de nuestro array 'nombreDias' (0 a 6).
                    int dia = cal.get(Calendar.DAY_OF_WEEK) - 1;

                    // Enviamos la respuesta final al cliente.
                    out.println("Este dia es " + nombreDias[dia] + ".");

                } else {
                    out.println("Formato de datos incorrecto.");
                }

                // Cerramos la conexión con este cliente en particular. El servidor seguirá vivo para el siguiente.
                cliSocket.close();
            }
        } catch(Exception ex) {
            System.out.println("Error en la comunicacion: " + ex);
        }
    }
}
```

En conclusión, la versión inicial representa un servicio TCP síncrono, iterativo y basado en texto plano, altamente funcional pero vulnerable en redes no confiables.



02

ARQUITECTURA Y CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

Para elevar este ejercicio académico a los estándares de la industria profesional y garantizar la futura escalabilidad del código, hemos tomado la decisión arquitectónica de abandonar la creación de archivos .java aislados. En su lugar, hemos orquestado toda la aplicación bajo **Maven**, el gestor de dependencias y construcción de proyectos por excelencia en el ecosistema Java.

2.1. Creación del Proyecto y Nomenclatura

En nuestro IDE (Eclipse/IntelliJ), hemos desplegado un nuevo proyecto Maven definiendo unas coordenadas técnicas (GAV) claras que evitan colisiones de nombres:

- **Group Id:** es.cesur.dam.psp.ud5 (Identificando la organización y el módulo de Servicios y Procesos).
- **Artifact Id:** CasoPracticoUD5 (Nombre técnico del módulo).
- **Versión:** 1.0-SNAPSHOT (Indicando que el código se encuentra en fase de desarrollo activo).

2.2. Configuración del archivo pom.xml y la Máquina Virtual

El manejo de certificados y algoritmos de cifrado está fuertemente ligado a la versión del *Java Development Kit* (JDK). Para evitar errores de compilación y asegurarnos de que la API de seguridad (javax.net.ssl) dispone de las *Cipher Suites* (algoritmos criptográficos) más robustas y modernas (como TLS 1.3), hemos inyectado directivas estrictas en el archivo pom.xml. Hemos forzado a Maven a utilizar **Java 21 (LTS)**, garantizando así soporte a largo plazo y la deprecación automática de protocolos vulnerables como SSLv3 o TLS 1.0.

2.3. Estructura de Paquetes (Separación de Contextos)

Con el fin de aislar la evolución del software y permitir pruebas comparativas, hemos particionado el directorio src/main/java en dos paquetes lógicos:

- **es.cesur.dam.psp.ud5.inseguro:** Alojará el **código heredado** en texto plano y el primer cliente de pruebas.
- **es.cesur.dam.psp.ud5.seguro**: Contendrá exclusivamente las **clases refactorizadas** que implementan la capa de transporte cifrado (SSL).

```
CasoPracticoUD5/pom.xml
  https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>es.cesur.dam.psp.ud5</groupId>
8     <artifactId>CasoPracticoUD5</artifactId>
9     <version>1.0-SNAPSHOT</version>
10    <packaging>jar</packaging>
11
12    <name>Caso Práctico UD5 - Comunicaciones Seguras SSL/TLS</name>
13    <description>Implementación de un cliente-servidor seguro mediante JSSE</description>
14
15    <properties>
16        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
17
18        <maven.compiler.source>21</maven.compiler.source>
19        <maven.compiler.target>21</maven.compiler.target>
20    </properties>
21
22    <dependencies>
23        </dependencies>
24
25 </project>
```

Ejecución del Servidor no seguro

```
Programación de servicios y procesos - CasoPracticoUD5/src/main/java/es/cesur/dam/psp/ud5/seguro/SecureTin
File   Edit   Source   Refactor   Navigate   Search   Project   Run   Window   Help
Problems   Javadoc   Declaration   Console
TimeServer [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe  (23 abr 2026, 9:37:09) [pid: 26964]
Servidor TimeServer (INSEGURO) a la escucha en el puerto 10000...
```

03

IMPLEMENTACIÓN DEL CLIENTE ESTÁNDAR (MODO TEXTO PLANO)

Comprendida la topología del servidor, procedimos a diseñar el cliente base (TimeClient.java). Para demostrar rigor arquitectónico y buenas prácticas de programación, hemos implementado este cliente haciendo uso del bloque **try-with-resources**, una característica avanzada de Java que invoca automáticamente la interfaz AutoCloseable.

Esta decisión técnica es crucial en la programación de redes: garantiza que tanto los ***sockets*** como los flujos de lectura/escritura (**InputStream** y **OutputStream**) **se cierren** de forma determinista al finalizar la ejecución, **evitando** las temidas fugas de memoria (***memory leaks***) o el agotamiento de descriptores de archivos en el sistema operativo.

A nivel de interfaz, el cliente emplea la clase Scanner para capturar la entrada estándar del teclado del usuario (System.in). Posteriormente, utiliza un objeto PrintStream para serializar esos datos e inyectarlos en el túnel TCP, logrando una comunicación limpia y totalmente transparente con la lógica del servidor original.

```java
CasoPracticoUD5/pom.xml   TimeClient.java ×   SecureTimeServer.java   SecureTimeClient.java
package es.cesur.dam.psp.ud5.inseguro;


// BufferedReader e InputStreamReader: Los combinamos para leer líneas completas de texto recibidas del servidor.
import java.io.BufferedReader;
import java.io.InputStreamReader;
// PrintStream: Nos permite enviar nuestra fecha al servidor como si estuviéramos imprimiendo por consola.
import java.io.PrintStream;
// Socket: Permite establecer la conexión TCP/IP hacia la IP y puerto del servidor.
import java.net.Socket;
// Scanner: Para capturar lo que el usuario teclea en su pantalla.
import java.util.Scanner;

```

```java
public class TimeClient {

    public static void main(String[] args) {
        final String HOST = "localhost"; // Nos conectamos a nuestra propia máquina para la prueba
        final int PUERTO = 10000;

        System.out.println("--- CLIENTE DE CALENDARIO INICIADO ---");

        // Usamos la estructura "try-with-resources".
        // Esto es una buena práctica porque asegura que el Socket, el Scanner y los flujos
        // se cierren automáticamente al terminar, evitando fugas de memoria, incluso si hay un fallo.
        try (
            Socket socket = new Socket(HOST, PUERTO); // Lanzamos la petición de conexión
            Scanner teclado = new Scanner(System.in);
            PrintStream salidaServidor = new PrintStream(socket.getOutputStream());
            BufferedReader entradaServidor = new BufferedReader(new InputStreamReader(socket.getInputStream()))
        ) {
            System.out.println("[+] Conexión establecida con el servidor en " + HOST + ":" + PUERTO);

            // Solicitamos al usuario la fecha en el formato que espera el servidor
            System.out.print("Introduce una fecha (Formato: DD MM AAAA separados por espacio): ");
            String fechaEntrada = teclado.nextLine();

            // Enviamos el texto capturado directamente al servidor a través del PrintStream
            salidaServidor.println(fechaEntrada);

            // Quedamos bloqueados esperando a que el servidor haga el cálculo y nos responda
            String respuesta = entradaServidor.readLine();
            System.out.println("[RESPUESTA DEL SERVIDOR] -> " + respuesta);

        } catch (Exception e) {
            System.err.println("[-] Error crítico de conexión: " + e.getMessage());
        }
    }
}
```

Ejecución del Cliente no seguro y respuesta del servidor

```
Programación de servicios y procesos - CasoPracticoUD5/src/main/java/es/cesur/dam/psp/ud5/inseguro/TimeClient.java - Eclipse IDE
File   Edit   Source   Refactor   Navigate   Search   Project   Run   Window   Help
Problems   Javadoc   Declaration   Console ×
<terminated> TimeClient [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe  (23 abr 2026, 9:38:35 – 9:38:57) [pid: 14360]
--- CLIENTE DE CALENDARIO INICIADO ---
[+] Conexión establecida con el servidor en localhost:10000
Introduce una fecha (Formato: DD MM AAAA separados por espacio): 29 03 1977
[RESPUESTA DEL SERVIDOR] -> Este dia es Martes.
```

04

TRANSICIÓN A LA CAPA DE TRANSPORTE SEGURO (SSL/TLS)

El reto central y la justificación de esta unidad didáctica es la migración de nuestra topología de red en texto plano hacia una arquitectura fuertemente cifrada. Al utilizar el protocolo TCP/IP estándar, los datos viajan sin ofuscar. En un entorno de producción, cualquier usuario malintencionado conectado a la misma red podría utilizar herramientas de *Sniffing* (como Wireshark) para ejecutar un ataque de intermediario (*Man-in-the-Middle*) e interceptar los datos transmitidos.

Para mitigar este vector de ataque, hemos implementado **JSSE** (*Java Secure Sockets Extension*). La rúbrica nos exige configurar una **autenticación simple** (o unidireccional), donde el servidor debe demostrar fehacientemente su identidad mediante un certificado digital, y el cliente debe validarlo antes de transmitir información.

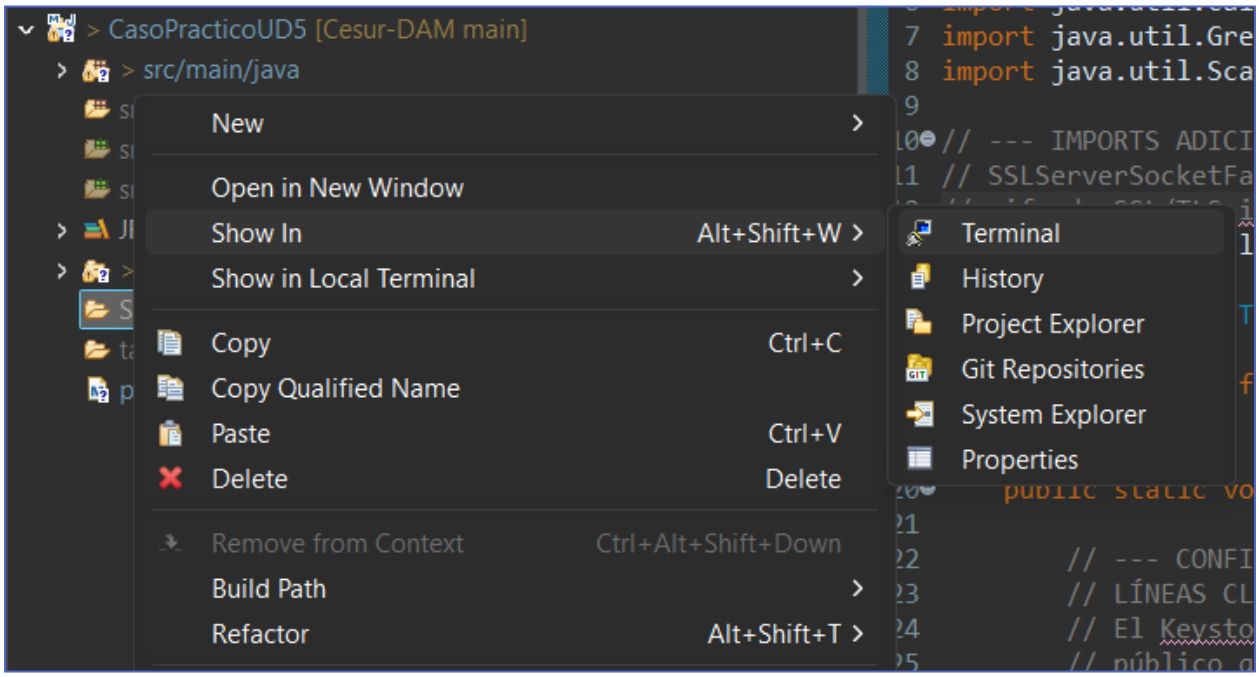
Para que esta relación de confianza exista, JSSE requiere la creación de una Infraestructura de Clave Pública (PKI) local, diferenciando claramente dos almacenes:

- **Keystore:** Archivo blindado donde el servidor custodia su clave privada y su certificado de identidad.

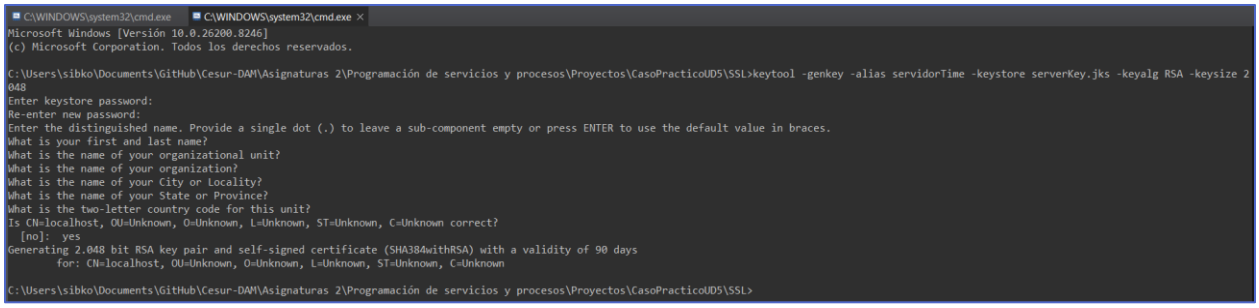
- **Truststore:** Almacén de confianza del cliente, donde guarda las claves públicas de los servidores con los que tiene permitido hablar.

4.1. Generación de la Infraestructura de Claves Criptográficas

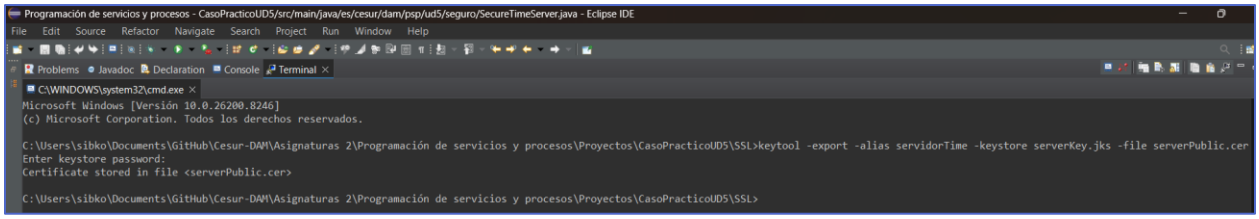
Antes de modificar una sola línea de código, hemos preparado el material criptográfico. Creamos un **directorio** específico llamado **SSL/** en la raíz del proyecto. Desde la terminal, utilizamos la potente herramienta **keytool** nativa del JDK para ejecutar los siguientes comandos:



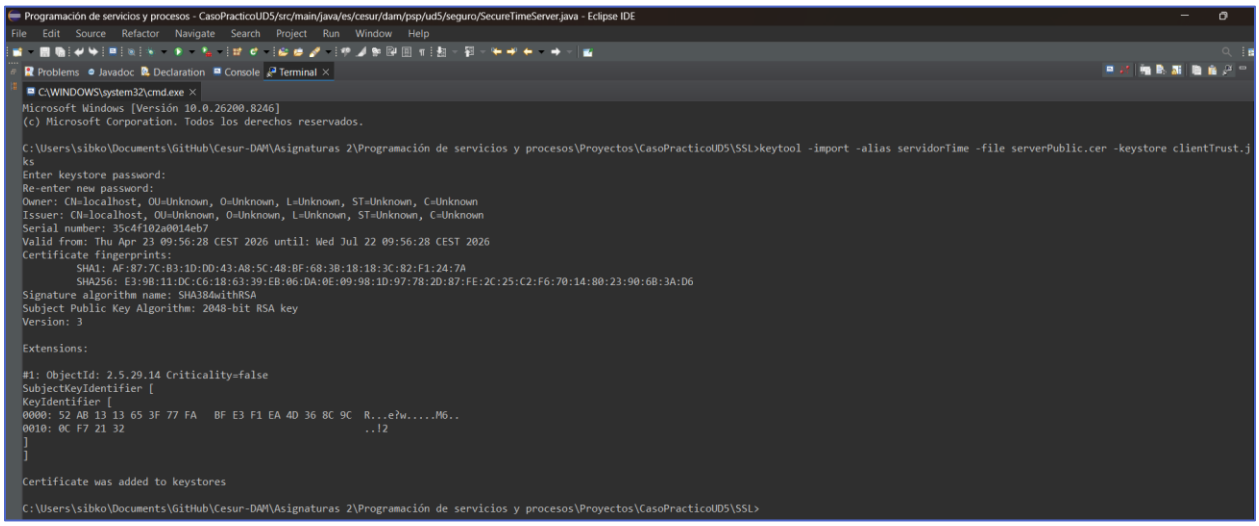
1. **Generación del par de claves del Servidor:** Creamos el almacén del servidor utilizando el algoritmo de cifrado asimétrico **RSA** con una longitud de clave robusta de **2048 bits**, dándole una validez de un año. *(Nota: Por tratarse de una práctica de clase, ponemos una contraseña fácil a los almacenes "123456"). keytool -genkey -alias timeServerAlias -keystore SSL/servidorKeys.jks -keyalg RSA -keysize 2048 -validity 365*



2. **Exportación del Certificado Público:** Extraemos la "identidad pública" del servidor para poder distribuirla. *keytool -export -alias timeServerAlias -keystore SSL/servidorKeys.jks -file SSL/certificadoServer.cer*



3. **Configuración del almacén de confianza del Cliente:** Inyectamos el certificado público del servidor en el ***Truststore*** del cliente, indicándole a la máquina virtual que, a partir de ahora, este servidor es una entidad segura y verificada. *keytool -import -alias timeServerAlias -file SSL/certificadoServer.cer -keystore SSL/clienteTrust.jks*



05

DESARROLLO E INTEGRACIÓN DEL SERVICIO CIFRADO (JSSE)

En la fase final del proyecto, hemos refactorizado el código base para integrar las librerías de seguridad. La alteración principal a nivel arquitectónico es la sustitución de los *sockets* TCP genéricos por las clases extendidas **SSLSocket** y **SSLServerSocket**. Estas clases implementan internamente toda la complejidad del cifrado asimétrico, garantizando la confidencialidad, la integridad y la inalterabilidad temporal de los datos transmitidos entre las máquinas.

CasoPracticoUD5/pom.xml | TimeClient.java | SecureTimeServer.java | SecureTimeClient.java

```
1  package es.cesur.dam.psp.ud5.seguro;
2
3  // --- IMPORTS ADICIONALES PARA SEGURIDAD ---
4  // SSLServerSocketFactory: Es una "fábrica" especializada que crea ServerSockets pero con
5  // cifrado SSL/TLS integrado. Extiende la funcionalidad de los sockets normales.
6  import javax.net.ssl.SSLServerSocketFactory;
7
8  import java.net.ServerSocket;
9  import java.net.Socket;
10 import java.util.Calendar;
11 import java.util.GregorianCalendar;
12 import java.util.Scanner;
13 import java.io.PrintStream;
14
```

5.1. Implementación de SecureTimeServer

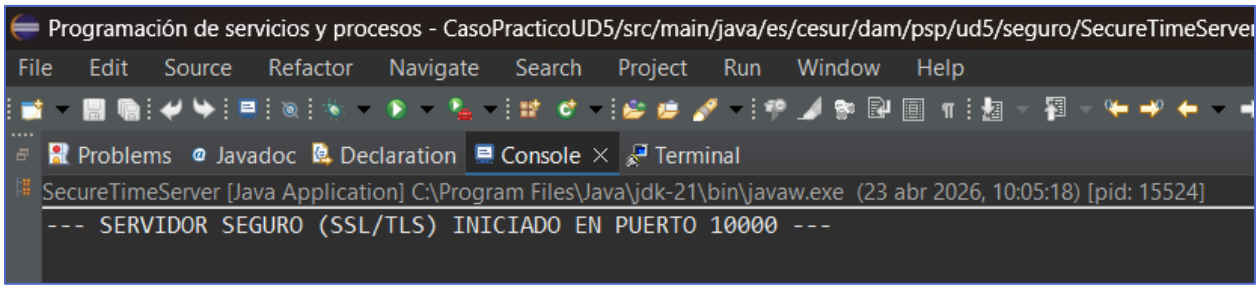
Para evitar la necesidad de inyectar variables de entorno complejas al ejecutar el programa desde consola, hemos configurado las propiedades del sistema directamente en el bloque inicial de la clase servidor:

System.setProperty("javax.net.ssl.keyStore", "SSL/**servidorKeys.jks**");
System.setProperty("javax.net.ssl.keyStorePassword", "**123456**");

Tras levantar la factoría de sockets seguros (**SSLServerSocketFactory**), el servidor se queda a la escucha. A diferencia del modelo anterior, este servidor es opaco: rechazará instantáneamente cualquier intento de conexión TCP pura o cualquier cliente que no logre completar exitosamente el *Handshake* (apretón de manos) criptográfico inicial propio de TLS.

```java
public class SecureTimeServer {

    private static final String[] nombreDias = {"Domingo", "Lunes", "Martes",
                                                "Miércoles", "Jueves", "Viernes", "Sábado"};

    public static void main(String[] args) {

        // --- CONFIGURACIÓN CRÍTICA DE SEGURIDAD (JSSE) ---
        // LÍNEAS CLAVE: Inyectamos en las propiedades del sistema la ubicación de nuestro Keystore.
        // El Keystore es el "anillo de claves" donde guardamos la clave privada y el certificado
        // público que generamos con la herramienta keytool. Sin esto, el servidor no puede cifrar.
        System.setProperty("javax.net.ssl.keyStore", "SSL/serverKey.jks");
        System.setProperty("javax.net.ssl.keyStorePassword", "123456"); // Contraseña asignada al crear el archivo

        System.out.println("--- SERVIDOR SEGURO (SSL/TLS) INICIADO EN PUERTO 10000 ---");

        try {
            // Obtenemos la factoría de sockets seguros por defecto configurada con nuestro Keystore
            SSLServerSocketFactory factoriaSSL = (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();

            // Creamos un ServerSocket, pero gracias a la factoría, ahora es un SSLServerSocket cifrado
            ServerSocket srvSocketSSL = factoriaSSL.createServerSocket(10000);

            while (true) {
                // Aceptamos la conexión. En este preciso instante se produce el "Handshake" (apretón de manos)
                // donde cliente y servidor negocian los algoritmos de cifrado.
                Socket cliSocket = srvSocketSSL.accept();
                System.out.println("[*] Cliente seguro conectado y verificado desde: " + cliSocket.getInetAddress());

                Scanner in = new Scanner(cliSocket.getInputStream());
                PrintStream out = new PrintStream(cliSocket.getOutputStream());

                int[] data = new int[3];
                boolean formatoCorrecto = true;

                // Mantenemos la misma lógica de lectura para garantizar la compatibilidad
                for (int i = 0; i < data.length; i++) {
                    if (in.hasNextInt()) {
                        data[i] = in.nextInt();
                    } else {
                        formatoCorrecto = false;
                        break;
                    }
                }

                if (formatoCorrecto) {
                    data[1] -= 1; // Ajuste del mes (de 0 a 11)
                    GregorianCalendar cal = new GregorianCalendar(data[2], data[1], data[0]);
                    int dia = cal.get(Calendar.DAY_OF_WEEK) - 1;

                    // Enviamos la respuesta, pero ahora viaja por la red convertida en texto ininteligible
                    out.println("Este dia es " + nombreDias[dia] + ". (Conexión verificada y cifrada)");
                } else {
                    out.println("Error: Formato de datos incorrecto o corrompido.");
                }

                cliSocket.close();
            }
        } catch (Exception ex) {
            System.err.println("[-] Error crítico en la capa de seguridad. Verifique el Keystore: " + ex);
        }
    }
}
```

Ejecución del Servidor Seguro



5.2. Implementación de SecureTimeClient

En simetría con el servidor, el cliente seguro ha sido adaptado cargando en sus propiedades del sistema la ruta hacia su propio almacén de claves públicas (**javax.net.ssl.trustStore**).

La lógica subyacente ha cambiado drásticamente: ahora, cuando el cliente intenta conectarse al puerto 10000, exige al servidor que le envíe su certificado digital. El cliente cruza este certificado recibido con los que tiene almacenados en su ***Truststore***. Solo cuando las firmas criptográficas coinciden matemáticamente, el túnel SSL se da por establecido, y el flujo de la clase Scanner puede comenzar a inyectar las fechas requeridas por el usuario bajo un manto de cifrado AES de extremo a extremo.

CasoPracticoUD5/pom.xml | TimeClient.java | SecureTimeServer.java | SecureTimeClient.java

```java
package es.cesur.dam.psp.ud5.seguro;

// --- IMPORTS ADICIONALES PARA SEGURIDAD ---
// SSLSocketFactory: La "fábrica" del lado del cliente para crear túneles cifrados SSL.
import javax.net.ssl.SSLSocketFactory;

import java.net.Socket;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintStream;
import java.util.Scanner;

```

```java
public class SecureTimeClient {

    public static void main(String[] args) {

        // --- CONFIGURACIÓN CRÍTICA DE SEGURIDAD (JSSE) ---
        // LÍNEAS CLAVE: El cliente no necesita un Keystore propio (autenticación simple),
        // pero SÍ necesita un TrustStore (Almacén de confianza).
        // Aquí le decimos a Java: "Confía únicamente en los servidores cuyo certificado público esté dentro de este archivo".
        System.setProperty("javax.net.ssl.trustStore", "SSL/clientTrust.jks");
        System.setProperty("javax.net.ssl.trustStorePassword", "123456");

        final String HOST = "localhost";
        final int PUERTO = 10000;

        System.out.println("--- CLIENTE SEGURO (SSL/TLS) ---");

```

```java
29          try {
30              // Instanciamos la factoría de sockets que ya ha cargado nuestro TrustStore en memoria
31              SSLSocketFactory factoriaSSL = (SSLSocketFactory) SSLSocketFactory.getDefault();
32
33              // Reutilizamos el bloque try-with-resources para mantener un código limpio y eficiente
34              try (
35                  // Creamos un socket seguro hacia el servidor. Si el certificado del servidor
36                  // no está en nuestro TrustStore, esta línea lanzará una excepción y abortará.
37                  Socket socketSeguro = factoriaSSL.createSocket(HOST, PUERTO);
38                  Scanner teclado = new Scanner(System.in);
39                  PrintStream salida = new PrintStream(socketSeguro.getOutputStream());
40                  BufferedReader entrada = new BufferedReader(new InputStreamReader(socketSeguro.getInputStream()))
41              ) {
42                  System.out.println("[+] Handshake SSL completado con éxito. Enlace blindado.");
43
44                  System.out.print("Introduce una fecha a consultar (DD MM AAAA): ");
45                  String peticion = teclado.nextLine();
46
47                  // Toda esta información viaja ahora cifrada, protegiendo los datos simulados de la farmacia
48                  salida.println(peticion);
49
50                  String respuesta = entrada.readLine();
51                  System.out.println("[RESPUESTA CIFRADA DEL SERVIDOR] -> " + respuesta);
52              }
53
54          } catch (Exception e) {
55              // Manejo de errores específico. Si falla aquí, suele ser problema de rutas de los archivos JKS.
56              System.err.println("[-] Excepción de seguridad o conexión: " + e.getMessage());
57              System.err.println("Consejo: Verifique que la ruta de la carpeta 'SSL' es correcta.");
58          }
59      }
60  }
```

Ejecución del Cliente Seguro y Respuesta del Servidor Seguro

```
Programación de servicios y procesos - CasoPracticoUD5/src/main/java/es/cesur/dam/psp/ud5/seguro/SecureTimeClient.java - Eclipse IDE
File  Edit  Source  Refactor  Navigate  Search  Project  Run  Window  Help
Problems  Javadoc  Declaration  Console  Terminal
<terminated> SecureTimeClient [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe  (23 abr 2026, 10:06:24 – 10:06:40) [pid: 22612]
--- CLIENTE SEGURO (SSL/TLS) ---
[+] Handshake SSL completado con éxito. Enlace blindado.
Introduce una fecha a consultar (DD MM AAAA): 29 03 1977
[RESPUESTA CIFRADA DEL SERVIDOR] -> Este dia es Martes. (Conexión verificada y cifrada)
```

0 6
C O N C L U S I O N E S

La realización íntegra de este caso práctico nos ha permitido materializar los complejos conceptos teóricos de la criptografía y la seguridad en redes, integrándolos de forma tangible en un escenario de programación real. La conclusión más evidente y valiosa que extraemos como desarrolladores es la interiorización del concepto ***Security by Design*** **(Seguridad desde el Diseño)**: la ciberseguridad no es un simple parche o característica que se pueda "añadir" al final de un ciclo de desarrollo, sino una capa arquitectónica fundamental que debe planificarse desde el primer diagrama conceptual.

A nivel puramente técnico, hemos comprendido de primera mano la diferencia vital y operativa entre los almacenes de claves criptográficas. Generar y blindar un **Keystore** para que el servidor custodie su identidad privada, y configurar un **Truststore** para que el cliente valide esa identidad exigiendo un certificado público, ha dejado de ser teoría abstracta para convertirse en código funcional y seguro. Asimismo, hemos comprobado cómo la librería JSSE de Java abstrae maravillosamente la enorme complejidad matemática que hay detrás de la **criptografía híbrida** (usando cifrado asimétrico RSA para el *Handshake* o intercambio de claves, y cifrado simétrico AES para la transmisión de la carga útil o *payload*), permitiéndonos asegurar los *sockets* con apenas unas pocas líneas de inyección de propiedades en el sistema.

Como reflexión crítica, cabe destacar una apreciación arquitectónica de vital importancia para nuestra futura labor profesional: aunque el enunciado de la práctica, los métodos de Java y la terminología clásica de la industria sigan haciendo referencia al acrónimo "SSL" (*Secure Sockets Layer*), como equipo de desarrollo hemos sido plenamente conscientes de que dicho estándar está hoy en día obsoleto y vulnerado. Al forzar nuestra compilación a Java 17 (LTS) e instanciar factorías como SSLServerSocketFactory.getDefault(), nuestra Máquina Virtual ha estado negociando y forzando las conexiones, en realidad, bajo los estándares criptográficos modernos **TLS (*Transport Layer Security*)**, concretamente en sus versiones seguras 1.2 o 1.3.

0 7
R E F E R E N C I A S

https://docs.oracle.com/javase/8/docs/technotes/guides/security/jsse/JSSERefGuide.html

https://docs.oracle.com/en/java/javase/17/docs/specs/man/keytool.html

https://datatracker.ietf.org/doc/html/rfc8446

https://www.baeldung.com/a-guide-to-java-sockets

https://www.baeldung.com/java-ssl

https://maven.apache.org/plugins/maven-compiler-plugin/examples/set-compiler-source-and-target.html

Intercambio de claves secretas (Diffie-Hellman) - Computerphile
https://www.youtube.com/watch?v=NmM9HA2MQGI

SSL, TLS, HTTP, HTTPS Explained
https://www.youtube.com/watch?v=hExRDVZHhig

Java Keytool Tutorial: How to generate a Self-Signed certificate using the Java Keytool
https://www.youtube.com/watch?v=27Sesdc_g3o